Chapter 4: Inter process communication(IPC)

- Inter process communication
- Race condition
- Critical section
- Mutual Exclusion
- Classic Problems

Inter process Communication

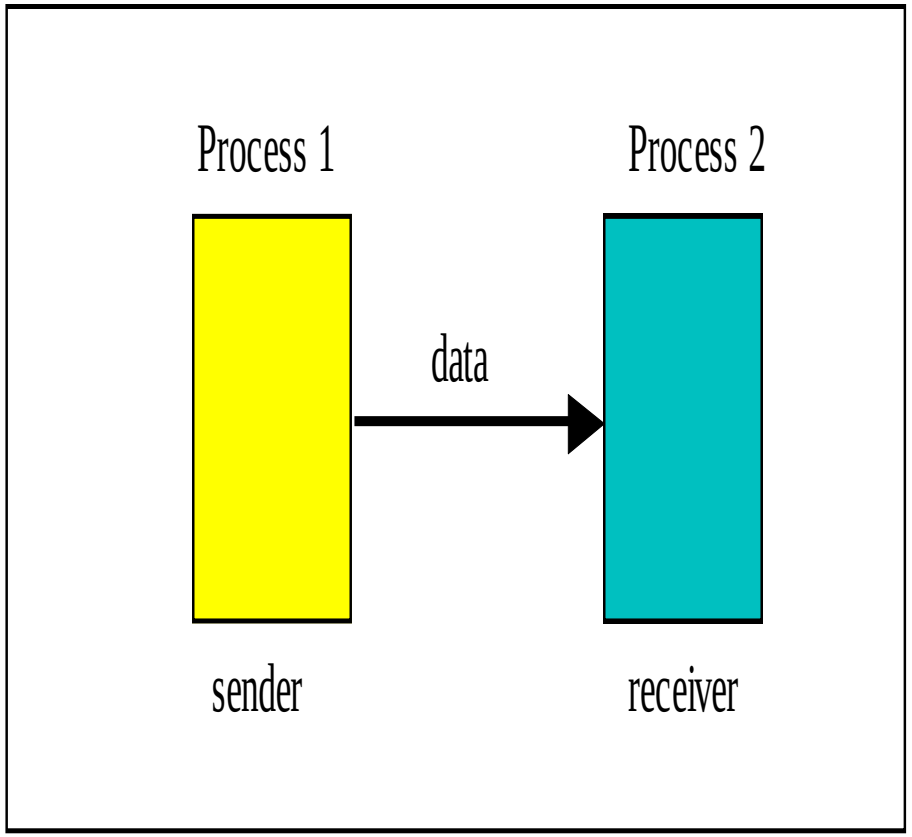
- The communication between processes in the control of the OS is called as *Inter-process Communication or simply IPC*.
- Processes frequently need to communicate with other processes/threads to share.
  - Messages
  - Semaphores
  - Shared memory

IPC- In distributed environment

Distributed computing systems make use of these facilities/resources to provide application programming interface (API) which allows IPC to be programmed at a higher level of abstraction.

(e.g., send and receive)

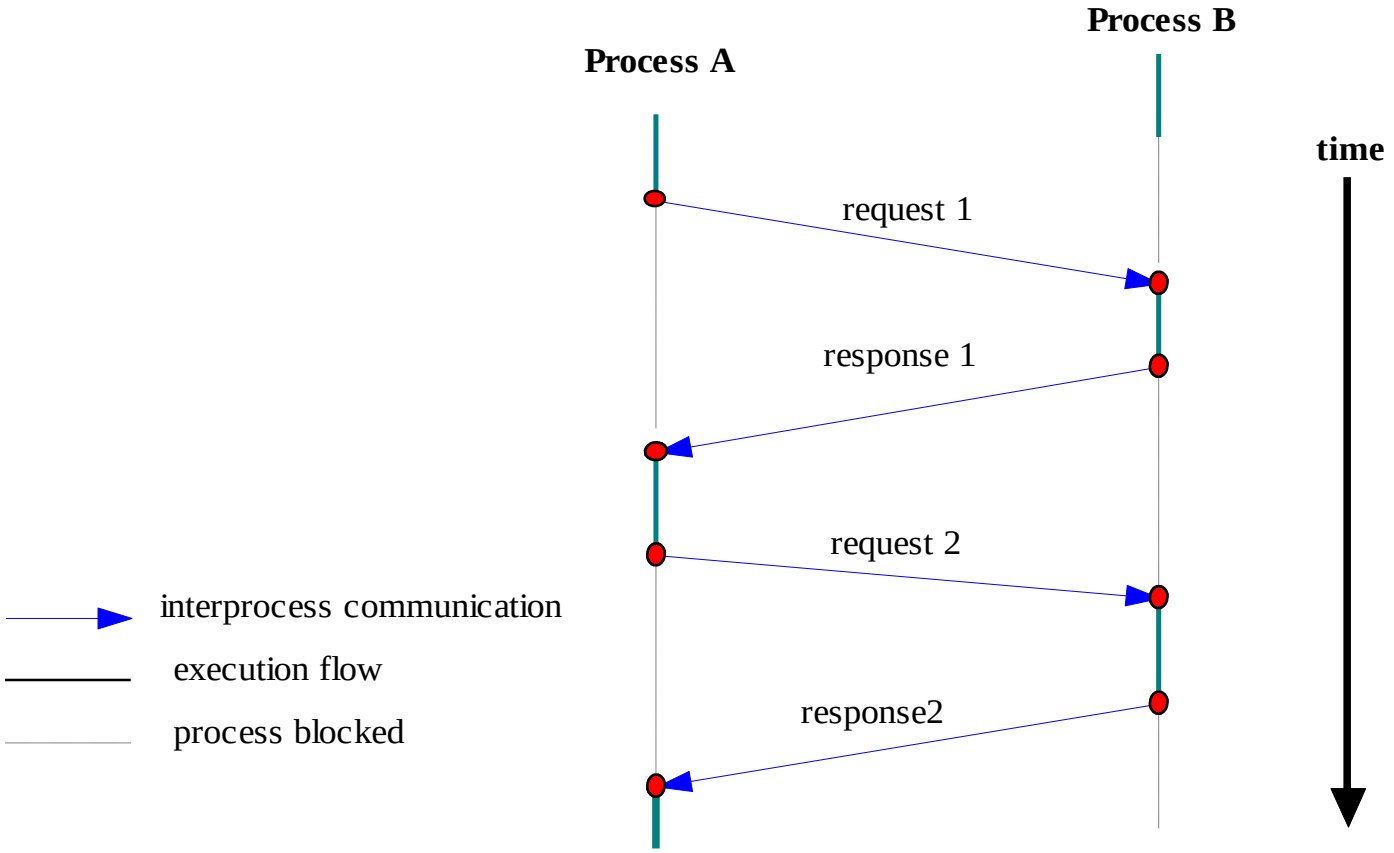
Distributed computing requires information to be exchanged among independent processes.



Operations provided in a standard Inter-process Communications as API

- **Connect** (sender address, receiver address),
  - For connection-oriented communication.
- **Send** ([receiver], message)
- **Receive** ([sender], message storage object)
- **Disconnect** (connection identifier),
  - For connection-oriented communication

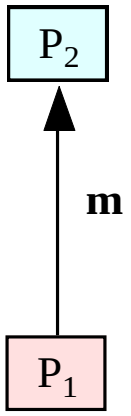
IPC – Event Diagram



Event diagram for a protocol

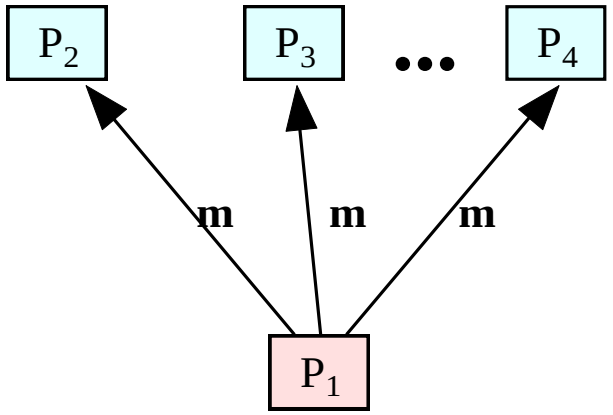
IPC – unicast and multicast

- In distributed computing, two or more processes engage in **IPC** using a **protocol** agreed upon by the processes.
- A process may be a sender at some points during a protocol, a receiver at other points.



unicast

Socket communication



multicast

Publish/Subscribe Message model,

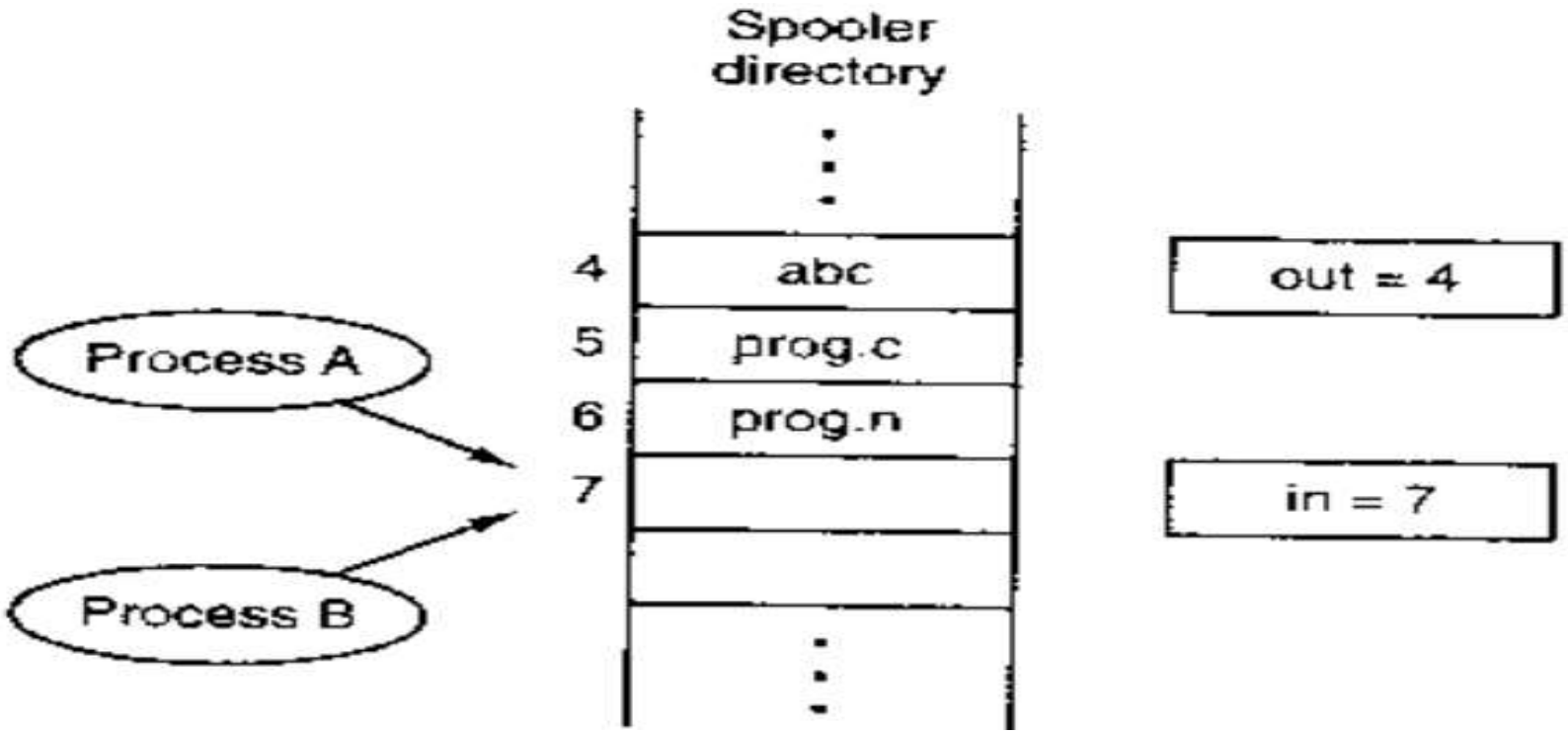
Why we need to study-IPC

- First – How one process can pass information to another.
  - APIs in distributed computing
- Second – conflict resolution when processes are engaging in critical activities.
  - Protocols and related issues.
- Third- proper sequencing when dependencies are present.
  - Data production and printing
- Two of the above issues are equally well works for threads and the first one is even more simple.
  - Same problems exist- same solutions apply

Race Condition

- Where two or more processes are reading or writing some shared data/shared memory and the final result depends on who runs precisely.
- It lead to unexpected behavior and data inconsistency in concurrent programs.
- To protect against the race condition, ensure that only one process at a time can be manipulating the shared data/variable.

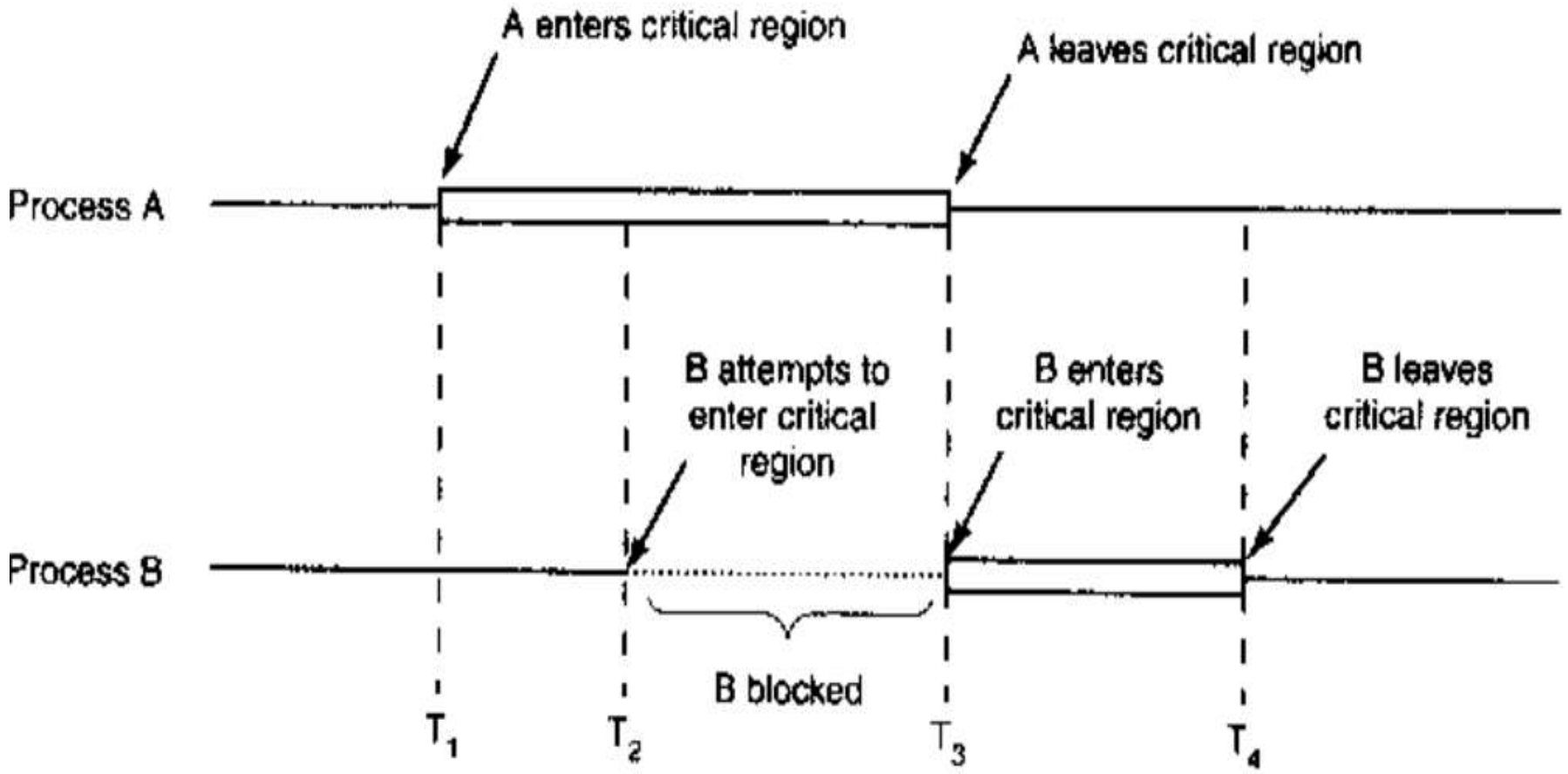
Race Condition



Critical Section

- Each process has a segment of code called **critical section (CS)** in which the process may be changing common variables, writing file and so on.
- If we could arrange matters such that **no two processes** were ever in their **critical sections at the same time**, we could avoid **race conditions**.
- Each process must request permission to enter its critical section i.e. **entry section**. The entry section may be followed by **exit section**. The remaining code is **remainder section**.
- Find some way to prohibit more than one process from reading and writing the shared data at the same time.(**mutual exclusion**).

Critical Section….



Mutual Exclusion

- A way of making sure that if one process is using a shared modifiable data, the other processes will be excluded from doing the same thing.
- Note that mutual exclusion needs to be enforced only when processes access shared modifiable data.
- Rather it needs to support concurrent programming.

Mutual Exclusion Conditions

1. No two processes may at the same moment inside their critical sections.
2. No process outside its critical section should block other processes.
3. No assumptions are made about relative speeds of processes or number of CPUs.
4. No process should wait arbitrary long to enter its critical section.

Proposals for Achieving mutual exclusion

1. Disabling Interrupts
2. Lock Variable
3. Strict Alternation
4. Peterson's algorithm
5. TSL instruction

Disabling Interrupts(Hardware Solution)

- Each process disables all interrupts just after entering in its critical section and re-enable all interrupts just before leaving critical section.
  - Has two drawbacks
    - Multi-processor
    - Re-enabling will not be performed
- Conclusion- good if it's done by the kernel itself

Lock Variables(software solution)

- Consider having a single, shared lock variable initialized to 0.
- When a process wants to enter its CS, it first tests the lock.
  - If the lock is 0, the process sets it to 1 and enters the CS.
  - If the lock is already 1, the process just waits until it becomes 0.

Lock Variables(software solution)

```
#define FALSE 0
#define TRUE   1
int lock=FALSE

PO:
{
    while (TRUE) {
      while (lock) { }; /* wait */
      lock=TRUE;
      CS( ) ;
      lock=FALSE;
      Non-CS( ) ;
    }
}
```

```
P1:
{
    while (TRUE) {
      while (lock) { }; /* wait */
      lock=TRUE;
      CS( ) ;
      lock=FALSE;
      Non-CS( ) ;
    }
}
```

Strict Alternation

- The integer variable 'turn',
- Processes inspect turn
  - E.g. *Process A inspect turn, finds it to be 0, and enters in its critical section. Process B also finds it to be 0 and sits in a loop continually testing 'turn' to see when it becomes 1.*
- Continuously testing a variable waiting for some value to appear is called the *Busy-Waiting*- (waste CPU time)
- A lock that uses busy waiting is spin lock.
- No process should allow to use the shared data in a successive manner.

Strict Alternation

```
int turn=0

PO:
{
   while (TRUE) {
       while (turn ! = 0)  { } ; /* wait */
       CS( ) ;
       turn = 1;
       Non-CS( ) ;
   }
}
```

```
P1:
{
   while (TRUE) {
     while (turn != 1) { } ; /* wait */
    CS( ) ;
    turn = 0;
    Non-CS( ) ;
   }
}
```

Petersons algorithm

- A Dutch mathematician (T. Dekker,1965) comes up with the solution of combining the idea of taking turns and lock variable.
  - It avoids strict alteration
- In 1981 G.L peterson comes with a simple algorithm to achieve mutual exclusion.
- Each process calls CS_*entry with its own process number, 0 or 1, as* parameter before entering to their cs.
- The process calls CS_*exit to indicate that it is done and to* allow the other process to enter.

```
...                                                  /* required header files */
#define FALSE 0
#define TRUE   1
#define N       2                                    /* number of processes */

int turn;                                            /* whose turn is it? */
int interested[N]                                    /* all elements initialized to FALSE */

void CS_entry (int process)                          /* process: Who is entering (0 or 1)? */
{
        int other;                                   /* number of the other process */
        other = 1 – process;                         /* the opposite of process */
        interested[process] = TRUE;                  /* show that you are interested */
        turn = process;                              /* set flag */
        while (turn == process) &&
                (interested[other] == TRUE )  { };   /* wait */
}

void CS_exit (int process)                           /* process: Who is leaving (0 or 1) ? */
{
        interested[process] = FALSE;                 /* indicate departure from CS */
}
```

Code for Process P_0:

```
do {
    flag[0] = true;
    turn = 1;
    while (flag[1] && turn == 1);
        // critical section
        flag[0] = false;
        // remainder section
} while (true);
```

Code for Process P_1:

```
do {
    flag[1] = true;
    turn = 0;
    while (flag[0] && turn == 0);
        // critical section
        flag[1] = false;
        // remainder section
} while (true);
```

- What if both processes claim to enter to their critical region almost simultaneously
- Needs busy waiting, hence CPU time wastage is there.

TSL instruction

- Proposed solution with a little help by the underling hardware.
- Used specially in multi-processor environment.
- Have an instruction TSL RX, LOCK – Test and Set lock
- It reads the content of the memory word ***lock*** into register RX. Then stores a nonzero value at the memory address lock.
  - This operation is guaranteed to be indivisible.
  - The CPU executing this instruction locks the memory bus in order not to be accessed by other processes.

TSL instruction….

```
enter_region:
TSL REGISTER,LOCK   |copy lock to register and set lock to 1
CMP REGISTER,#0     | was lock zero?
JNE enter_region    | if it was non zero, lock was set, so loop
RET                 | return to caller; critical region entered
leave_region:
MOVE LOCK,#0         | store a 0 in lock
RET                  | return to caller
```

Sleep and Wakeup

- Avoid priority inversion problem and CPU time wastage in TSL and Peterson's case.
- Uses a pair of sleep and wakeup.
  - Sleep is a system call that causes the caller to be sleep (suspended) until….
  - Wakeup call has one parameter, the process to be awakened.
  - Alternatively both sleep and wakeup use one parameter. A memory address that much up both sleeps and wakeups.

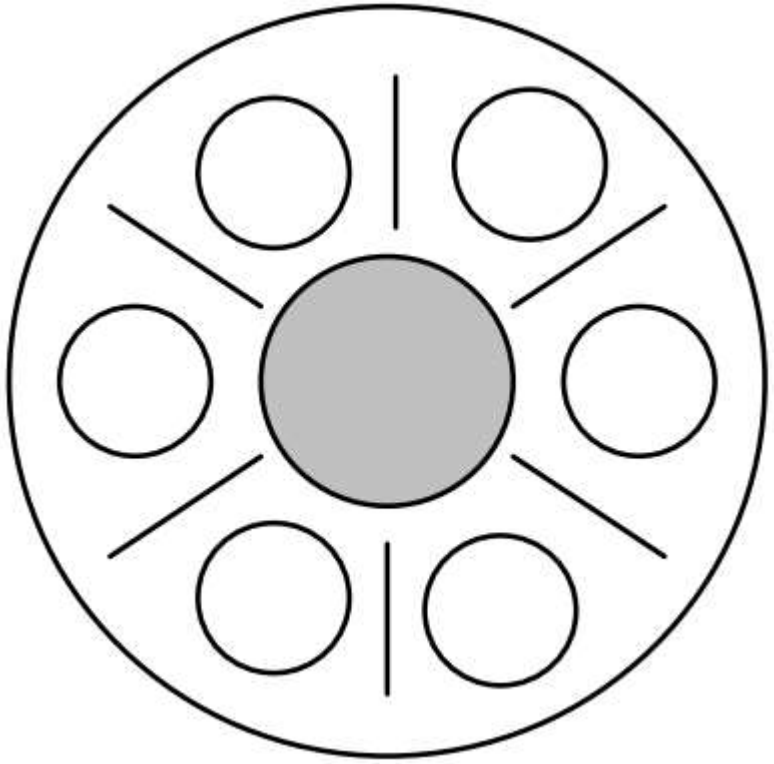
Producer- Consumer problem

- A.K.A. bounded-buffer problem.
- Two processes share a common fixed size buffer.
- Producer puts information in to the buffer
- Consumer takes out of information from the buffer.
- Race condition occurs
  - When the producer finds the buffer full
  - When the consumer finds the buffer empty
  - In such cases both processes will be put into sleep forever.

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
      int item;

      while (TRUE) {                            /* repeat forever */
            item = produce_item( );             /* generate next item */
            if (count == N) sleep( );           /* if buffer is full, go to sleep */
            insert_item(item);                  /* put item in buffer */
            count = count + 1;                  /* increment count of items in buffer */
            if (count == 1) wakeup(consumer);   /* was buffer empty? */
      }
}

void consumer(void)
{
      int item;

      while (TRUE) {                            /* repeat forever */
            if (count == 0) sleep( );           /* if buffer is empty, got to sleep */
            item = remove_item( );              /* take item out of buffer */
            count = count − 1;                  /* decrement count of items in buffer */
            if (count == N − 1) wakeup(producer); /* was buffer full? */
            consume_item(item);                 /* print item */
      }
}
```

The Dining Philosophers Problem

- There are N philosophers spending their lives thinking and eating in a room. In their round table there is a plate of infinite rice and N chopsticks.
- A philosopher that picks both chopsticks successfully (one at a time) starts eating.



The Dining Philosophers Problem

- When a philosopher finishes eating, he puts down both of his chopsticks to their original position and he starts thinking again.
- The question is to write a program which does not let any philosopher to die due to hunger (i.e. no deadlocks).

End of chapter 4